Name:- Jaykishan J Saharan
Div:- D Roll no:- 26
Course/Program:- BTech CSE SY
Subject:- DSA

DSA lab assignment 7,,

Q.1) Consider the infix expression (3*7)+10.

(a) Consider this expression to postfix using the infix→Postfix algorithm.

(b) Evaluate the postfix and show the stack contents.

(c) Give the final result and briefly justify each stack operation. Enlist the rules for infix to postfix expressions.

Sol^n:- Infix → Postfix

I/p	o/p	Stack.
(		
3	3	
*	3	
7	37	+
)	37*	(
+	37*	*
10	37*10+	&

⇒ Postfix: 37*10+

⇒ Postfix Evaluation:- 37*10+ = ?

1) Input:- 3 ⇒ Push 3 into stack.
2) Push 7 into the stack.
3) Input is *, which is an operator, we pop the operator from the stack, evaluate it and then we push it back to the stack. So,

operands = 7×3 = 21 ⇒ Push 21 onto stack.

21

4). Input 10, push into the stack.

10
21

5). Input is +, which is an operator, so we pop all operations and then push it back.

operands = 21+10 ⇒ 31 = Push into Stack.

31

∴ Postfix evaluation of 37+10+ is 31.

Rules for conversion of infix → postfix.

i. We have to scan expression from left → right.
ii. If input is operand, then append it to output.
iii. If input is '(', then push it into the stack.
iv. If input is ')', then pop all contents till '('
v. If input is operator, then push it into stack. but if of equal precedence, then we need to pop operators which are already on stack.
vi. when it reaches '\' (null character), then pop all of the stack and append it to output.
vii. Precedence order:- ^ → exponentiation.
*, / → multiplication, division
+, - → addition, subtraction.

Q.2) Write the algorithm to convert an infix expression to postfix expression using stack operations.

Sol^n :- 1) Initialize.
Create an empty operator stack.
Create an empty postfix string.
2). Scan the infix expression from left to right.

for each character c in the expression:
case 1: operand (A-Z, a-z, 0-9).
→ append c directly to postfix.
case 2: opening parenthesis '('.
→ Push in onto the stack.
case 3: closing parenthesis ')'
→ Pop operators from the stack and append to postfix until an opening parenthesis is found.
→ Discard the c.
case 4: operator (+, −, *, /, ^)
→ while the stack is not empty and the precedence of the top of stack is greater than or equal to that of c.
→ Pop operator from stack and append to postfix
→ Push c onto the stack.
After Scanning the entire expression:
Pop all remaining operators from the stack and append to postfix.
Return postfix expression.

write algorithm for infix → prefix expression.
Inpot the infix expression as a string.
Reverse the infix string.
Swap brackets: replace each '(' with ')' and replace.
call infix → Postfix on this modified expression
Reverse the obtained postfix expression.
output the result as the prefix expression.

Q.4) Apply the infix → postfix to convert: (A+B) / (C-D)

Sol^n :-

I/P	O/P	Stack.
(	-	
A	A	
+	A	
B	AB	(
)	AB+	-
/	AB+	/
(	AB+	/
(	AB+G	(
-	AB+C	*
D	AB+CD	/
)	AB+CD-	
/o	AB+CD-/	

⇒ Postfix Expression :- AB+CD-/

Q.5) Convert infix into Postfix and Prefix using rules of operator presecdence : (A+B) + (C-D)

Sol^n :- Infix → Postfix.

I/P	O/P	Stack.
(	-	
A	A	
+	A	
B	AB	(
)	AB+	+
*	AB+	/
(	AB+	*
(	AB+C	(
-	AB+C	*
D	AB+CD	/
)	AB+CD-	/
/o	AB+CD-*/	

∴ Postfix expression :- AB+cD-*

→ Infix to Prefix

1) Reverse the Input

(A+B) * (C-D) ⇒)D-C(*)B+A(

2) Swap brackets.

)D-C(*)B+A(⇒ (D-C)*(B+A)

3) This expression → Postfix : (D-c) * (B+A)

I/P	o/p	stack.
(	-	
D	D	
-	D	
C	DC	
)	DC-	✗
*	DC-	*
C	DC-	(
B	DC-B	+
+	DC-B	✗
A	DC-BA	✗
)	DC-BA+	(
/o	DC-BA+*	

⇒ Expression ⇒ DC-BA+*

4) Reverse this expression

DC-BA+* ⇒ *+AB-CD

* Conclusion: Thus, we have implemented a program to convert an expression from infix to postfix and infix to prefix.